

Manipuler des données avec Pandas : statistiques et restructuration

Lino Galiana

Version du 29 septembre 2026

Après avoir vu comment associer des jeux de données grâce à des jointures, ce chapitre poursuit l'exploration de la boîte à outils du *data scientist* avec deux opérations classiques : les statistiques descriptives par groupe et la restructuration de données (*reshape*). Il propose également une introduction à la construction de tableaux avec `great_tables`.

⚠ PDF généré automatiquement. Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Introduction	1
1.1 Données	2
2 Statistiques descriptives par groupe	3
2.1 Principe	3
2.2 Illustration 1: dénombrement par groupe	4
2.3 Illustration 2: agrégats par groupe	6
3 Exercice d'application	7
4 Restructurer les données	8
4.1 Principe	8
4.2 Exercice d'application	9
5 Formater des tableaux de statistiques descriptives	10
Informations additionnelles	13
Bibliographie	14

💡 Compétences à l'issue de ce chapitre

- Construire des statistiques agrégées fines grâce aux méthodes de `Pandas` ;
- Savoir restructurer ses données du format *long* au format *wide* (et inversement) ;
- Créer des tableaux attractifs pour communiquer sur des résultats agrégés.

1 Introduction

Le [chapitre précédent](#) a montré comment enrichir un jeu de données en l'associant à une autre source grâce à une jointure. Nous disposons désormais de deux jeux de données mis en cohérence: les émissions de gaz à effet de serre de l'Ademe (`emissions`) et les données de cadrage Filosofi de l'Insee (`filosofi`).

Ce chapitre va exploiter cette association pour approfondir notre compréhension des données grâce à deux opérations complémentaires:

- les statistiques descriptives par groupe ;

- la restructuration de données entre formats *long* et *wide*.

Nous verrons enfin comment construire, à partir de ces statistiques, des tableaux de communication soignés grâce au *package* `great_tables`.

1.1 Données

Nous repartons des mêmes jeux de données que le [chapitre sur les jointures](#): les émissions communales de l'Ademe (`emissions`) et les données Filosofi de l'Insee (`filosofi`), enrichies l'une par l'autre. Le code ci-dessous, déjà commenté en détail dans le chapitre précédent, permet de reconstituer ces jeux de données:

```
import requests
import pandas as pd

def get_melodi(dataset, dimension, code, value_name, time_period, geo_level="COM"):

    url = f"https://api.insee.fr/melodi/data/{dataset}"

    params = {
        "GEO": geo_level,
        "dimension": code,
        "TIME_PERIOD": time_period,
        "maxResult": 40000,
    }

    response = requests.get(url, params=params, timeout=60)
    response.raise_for_status()
    observations = response.json()["observations"]

    return pd.DataFrame(
        {
            "CODGEO": obs["dimensions"]["GEO"].split("-")[-1],
            value_name: obs["measures"]["OBS_VALUE_NIVEAU"].get("value"),
        }
        for obs in observations
    )

niveau_vie = get_melodi("DS_FILOSOFI_CC", "FILOSOFI_MEASURE", "MED_SL", "NIVVIE_MEDIAN", 2023)
taux_pauvrete = get_melodi("DS_FILOSOFI_CC", "FILOSOFI_MEASURE", "PR_MD60", "TAUX_PAUVRETE",
2023)
population = get_melodi("DS_POPULATIONS_HISTORIQUES", "POPREF_MEASURE", "PMUN", "POPULATION",
2023)

url_cog_2023 = "https://www.insee.fr/fr/statistiques/fichier/6800675/v_commune_2023.csv"
url_backup = "https://minio.lab.sspcloud.fr/lgaliana/data/python-ENSAE/cog_2023.csv"

try:
    cog_2023 = pd.read_csv(url_cog_2023)
except requests.exceptions.Timeout:
    cog_2023 = pd.read_csv(url_backup)

noms_communes = (
    cog_2023
    .loc[cog_2023["TYPECOM"] == "COM", ["COM", "LIBELLE"]]
    .rename(columns={"COM": "CODGEO", "LIBELLE": "LIBGEO"})
)
```

```

filosofi = (
    niveau_vie
    .merge(taux_pauvrete, on="CODGEO", how="outer")
    .merge(population, on="CODGEO", how="outer")
    .merge(noms_communes, on="CODGEO", how="left")
    [["CODGEO", "LIBGEO", "POPULATION", "NIVVIE_MEDIAN", "TAUX_PAUVRETE"]])

filosofi["POPULATION"] = filosofi["POPULATION"].astype(int)
filosofi["dep"] = filosofi["CODGEO"].str[:2]

```

```

import pandas as pd

url = "https://data.ademe.fr/data-fair/api/v1/datasets/igt-pouvoir-de-rechauffement-global/convert"
emissions = pd.read_csv(url)
emissions["dep"] = emissions["INSEE commune"].str[:2]

emissions.head(2)

```

	INSEE commune	Commune	Agriculture	Autres transports	Autres transports international	CO2 biomasse hors-total	Déchets	Energie	Industrie hors-énergie	Résidentiel	Routier	Tertiaire	dep
0	01001	L'ABERGEMENT-CLEMENCIAT	3711.425991	NaN	NaN	432.751835	101.430476	2.354558	6.911213	309.358195	793.156501	367.036172	01
1	01002	L'ABERGEMENT-DE-VAREY	475.330205	NaN	NaN	140.741660	140.675439	2.354558	6.911213	104.866444	348.997893	112.934207	01

Nous aurons également besoin des *packages* suivants

Pour obtenir des résultats reproductibles, on peut fixer la racine du générateur pseudo-aléatoire.

```
np.random.seed(123)
```

2 Statistiques descriptives par groupe

2.1 Principe

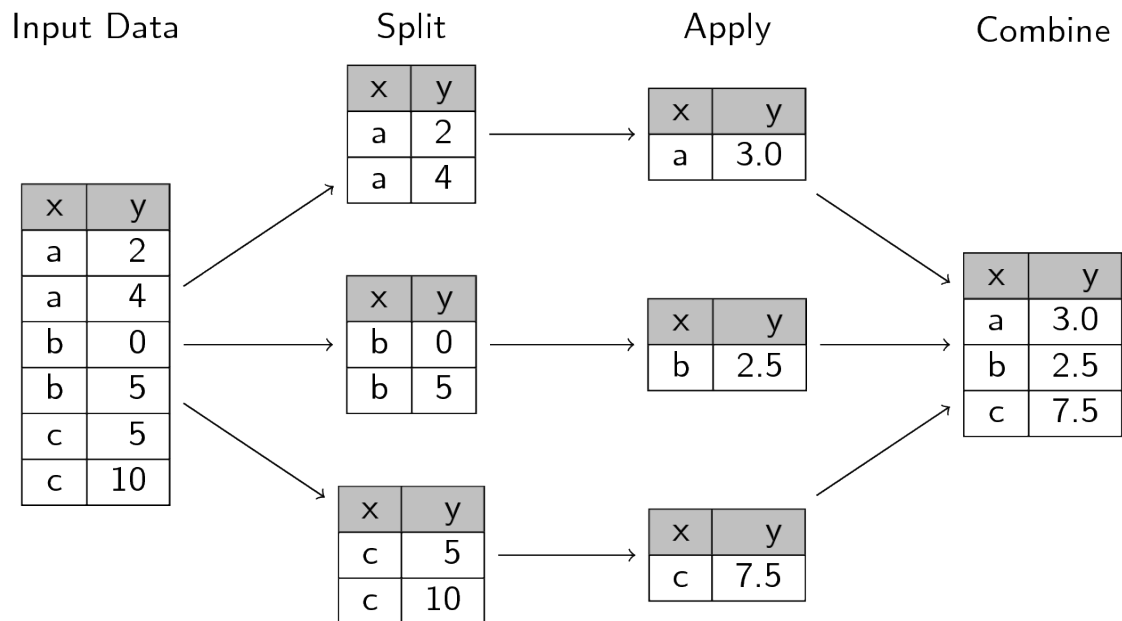
Nous avons vu, lors du chapitre d'introduction, comment obtenir une statistique agrégée simplement grâce à `Pandas`. Il est néanmoins commun d'avoir des données avec des strates intermédiaires d'analyse pertinentes: des variables géographiques, l'appartenance à des groupes socio-démographiques liés à des caractéristiques renseignées, des indicatrices de période temporelle, etc. Pour mieux comprendre la structure de ses données, les *data scientists* sont donc souvent amenés à construire des statistiques descriptives sur des sous-groupes présents dans les données.

Pour reprendre l'exemple sur les émissions, nous avons précédemment construit des statistiques d'émissions au niveau national. Mais qu'en est-il du profil d'émission des différents départements ? Pour répondre à cette question, il sera utile d'agréger nos données au niveau départemental. Ceci nous donnera une information différente du jeu de données initial (niveau communal) et du niveau le plus agrégé (niveau national).

En `SQL`, il est très simple de découper des données pour effectuer des opérations sur des blocs cohérents et recollecter des résultats dans la dimension appropriée.

La logique sous-jacente est celle du *split-apply-combine* qui est repris par les langages de manipulation de données, auxquels `pandas` ne fait pas exception.

L'image suivante, issue de [ce site](#), représente bien la manière dont fonctionne l'approche `split-apply-combine`:

Figure 1. – Split-apply-combine (Source: unlhcc.github.io)

En **Pandas**, on utilise **groupby** pour découper les données selon un ou plusieurs axes (ce [tutoriel](#) sur le sujet est particulièrement utile). L'ensemble des opérations d'agrégation (comptage, moyennes, etc.) que nous avons vues précédemment peut être mise en oeuvre par groupe.

Techniquement, cette opération consiste à créer une association entre des labels (valeurs des variables de groupe) et des observations. Utiliser la méthode **groupby** ne déclenche pas d'opérations avant la mise en oeuvre d'une statistique, cela crée seulement une relation formelle entre des observations et des regroupements qui seront utilisés *a posteriori*:

```
filosofi.groupby('dep').__class__
```

```
pandas.api.typing.DataFrameGroupBy
```

Tant qu'on n'appelle pas une action sur un **DataFrame** par groupe, du type **head** ou **display**, **pandas** n'effectue aucune opération. On parle de *lazy evaluation*. Par exemple, le résultat de `df.groupby('dep')` est une transformation qui n'est pas encore évaluée :

```
filosofi.groupby('dep')
```

```
<pandas.api.typing.DataFrameGroupBy object at 0x7ff25c841590>
```

2.2 Illustration 1: dénombrement par groupe

Pour illustrer l'application de ce principe à un comptage, on peut dénombrer le nombre de communes par département en 2023 (chaque année cette statistique change du fait des fusions de communes). Pour cela, il suffit de prendre le référentiel des communes françaises issu du code officiel géographique (COG) et dénombrer par département grâce à **count**:

Grâce à ce jeu de données, sans avoir recours aux statistiques par groupe, on peut déjà savoir combien on a, respectivement, de communes, départements et régions en France:

```
communes = cog_2023.loc[cog_2023['TYPECOM']=="COM"]
communes.loc[:, ['COM', 'DEP', 'REG']].nunique()
```

Ligne 1

On se restreint au statut « Commune » car ce fichier comporte également les codes Insee pour d'autres status, comme les « Arrondissements municipaux » de Paris, Lyon et Marseille.

```
COM      34945
DEP       101
REG        18
dtype: int64
```

Maintenant, intéressons nous aux départements ayant le plus de communes. Il s'agit de la même fonction de dénombrement où on joue, cette fois, sur le groupe à partir duquel est calculé la statistique.

Calculer cette statistique se fait de manière assez transparente lorsqu'on connaît le principe d'un calcul de statistiques avec **Pandas**:

```
communes = cog_2023.loc[cog_2023['TYPECOM']=="COM"]
communes.groupby('DEP').agg({'COM': 'nunique'})
```

	COM
DEP	
01	392
02	798
03	317
04	198
05	162
...	...
971	32
972	34
973	22
974	24
976	17

En SQL, on utiliserait la requête suivante:

```
SELECT dep, COUNT DISTINCT "COM" AS COM
FROM communes
GROUP BY dep
WHERE TYPECOM = 'COM';
```

La sortie est une **Series** indexée. Ce n'est pas très pratique comme nous avons pu l'évoquer au cours du chapitre précédent. Il est plus pratique de transformer cet objet en **DataFrame** avec **reset_index**. Enfin, avec **sort_values**, on obtient la statistique désirée:

```
(
    communes
    .groupby('DEP')
    .agg({'COM': 'nunique'})
    .reset_index()
```

```
.sort_values('COM', ascending = False)
)
```

	DEP	COM
62	62	890
1	02	798
80	80	772
57	57	725
76	76	708
...	...	...
96	971	32
99	974	24
98	973	22
100	976	17
75	75	1

2.3 Illustration 2: agrégats par groupe

Pour illustrer les agrégats par groupe nous pouvons prendre le jeu de données de l'Insee `filosofi` et sommer la variable `POPULATION`.

Pour calculer le total au niveau France entière nous pouvons faire de deux manières :

```
filosofi['POPULATION'].sum()* 1e-6
```

```
np.float64(68.09428)
```

```
filosofi.agg({"POPULATION": "sum"}).div(1e6)
```

```
POPULATION    68.09428
dtype: float64
```

où les résultats sont reportés en millions de personnes. La logique est identique lorsqu'on fait des statistiques par groupe, il s'agit seulement de remplacer `filosofi` par `filosofi.groupby('dep')` pour créer une version partitionnée par département de notre jeu de données:

```
filosofi.groupby('dep')['POPULATION'].sum()
```

Ligne 1

Avec cette approche, il faut faire attention à l'ordre des opérations: d'abord on effectue le `groupby` puis on conserve la colonne d'intérêt

```
dep
01    679344
02    523342
03    333298
04    168054
05    143467
```

```
...
92    1654712
93    1704316
94    1426929
95    1281653
97    1928465
Name: POPULATION, Length: 97, dtype: int64
```

```
filosofi.groupby('dep').agg({"POPULATION": "sum"})
```

POPULATION	
dep	
01	679344
02	523342
03	333298
04	168054
05	143467
...	...
92	1654712
93	1704316
94	1426929
95	1281653
97	1928465

La seconde approche est plus pratique car elle donne directement un `DataFrame Pandas` et non une série indexée sans nom. A partir de celle-ci, quelques manipulations basiques peuvent suffire pour avoir un tableau diffusables sur la démographie départementale. Néanmoins, celui-ci, serait quelques peu brut de décoffrage car nous ne possédons à l'heure actuelle que les numéros de département. Pour avoir le nom de départements, il faudrait utiliser une deuxième base de données et croiser les informations communes entre elles (en l'occurrence le numéro du département) : c'est le principe des jointures que nous avons mis en oeuvre dans le [chapitre précédent](#).

3 Exercice d'application

Cet exercice d'application s'appuie sur le jeu de données de l'Ademe nommé `emissions` précédemment.

💡 Exercice 1 : agrégations par groupe

1. Calculer les émissions totales du secteur « Résidentiel » par département et rapporter la valeur au département le plus polluant dans le domaine. En tirer des intuitions sur la réalité que cette statistique reflète.
2. Calculer, pour chaque département, les émissions totales de chaque secteur. Pour chaque département, calculer la proportion des émissions totales venant de chaque secteur.

Indices pour cette question

- « Grouper par » = `groupby`

```
• « émissions totales » = agg({'**' : "sum"})
```

A la question 1, le résultat obtenu devrait être le suivant:

	dep	Résidentiel	Résidentiel (% valeur max)
59	59	3.498347e+06	1.000000
75	75	1.934580e+06	0.552998
69	69	1.774653e+06	0.507283
62	62	1.738090e+06	0.496832
57	57	1.644192e+06	0.469991

Ce classement reflète peut-être plus la démographie que le processus qu'on désire mesurer. Sans l'ajout d'une information annexe sur la population de chaque département pour contrôler ce facteur, on peut difficilement savoir s'il y a une différence structurelle de comportement entre les habitants du Nord (département 59) et ceux de la Moselle (département 57).

A l'issue de la question 2, prenons la part des émissions de l'agriculture et du secteur tertiaire dans les émissions départementales:

dep	Agriculture	Autres transports	Autres transports international	CO2 biomasse hors total	Déchets	Energie	Industrie hors énergie	Résidentiel	Routier	Tertiaire	Part Agriculture	Part Autres transports	Part Autres transports international	Part CO2 biomasse hors total	Part Déchets	Part Energie	Part Industrie hors énergie	Part Résidentiel	Part Routier	Part Tertiaire
23	1.43000e+06	506057001	0.000000	230196.604309	26550.838041	9752578164	28626.240499	134097.315356	43467.608075	70731.240013	66.855172	0.213526	0.000000	6.904716	1.129866	0.415012	1.218163	5.710847	18.501132	3.009086
48	7.51090e+05	509719812	0.000000	70905.948092	20111.391018	6063540742	17805.203591	61053.909393	23018.408432	43061.121159	66.772448	0.000000	0.000000	5.737238	2.104764	0.409761	1.409164	4.998405	20.521701	3.532047
19	1.39000e+06	6201214100	0.000000	280161.900777	40614.670202	12178.402196	82021.400184	120151.601994	640021.901018	84064.610185	30.761414	0.200777	0.000730	8.600517	1.759968	0.310113	3.300560	4.000000	17.210364	3.270364
12	1.12253e+06	13796.08078	3224.040000	351.62018409	52102.681208	35405.604764	112057.400887	208062.654209	759451.903037	170211.843832	54.336487	0.313227	0.000004	8.405106	1.541893	0.906467	2.890437	6.003341	20.364540	4.957839
52	1.02600e+06	499942452	0.000000	201732.703762	50950.604526	10651.432366	53468.400055	150218.000190	640481.993580	105662.674213	49.732924	0.222835	0.000000	9.772766	2.466261	0.806664	2.930235	7.664754	21.622045	5.110737

dep	Agriculture	Autres transports	Autres transports international	CO2 biomasse hors total	Déchets	Energie	Industrie hors énergie	Résidentiel	Routier	Tertiaire	Part Agriculture	Part Autres transports	Part Autres transports international	Part CO2 biomasse hors total	Part Déchets	Part Energie	Part Industrie hors énergie	Part Résidentiel	Part Routier	Part Tertiaire
75	0.000000	42216.820025	1.83700e+02	1.186577e+01	27558.781206	147065.117371	434014.400384	1.93458e+06	1.62358e+06	1.331630e+06	0.000000	0.612725	0.002730	17.630002	0.406495	2.104847	6.453018	28.743070	24.152808	19.785275
94	2209.420451	218992.353559	3.16420e+05	3.16420e+05	213619.601516	76361.230740	467189.030727	1.330819e+06	1.309432e+06	7.630302e+05	0.049091	4.167701	5.907808	13.138562	4.860530	1.823898	8.891367	25.440273	22.236193	14.931005
92	91.440104	1206.794039	2.105170e+02	1.867090e+01	264497.000711	124284.100012	700397.400067	5.40012e+06	1.290420e+06	8.300112e+05	0.001177	0.212910	0.000025	16.420100	4.566691	4.100041	12.191512	25.500102	20.477565	14.642794
95	2018.470062	59627.086124	1.101600e+06	7.25919e+05	252168.907778	102037.607003	452316.500990	1.316163e+06	1.700191e+06	8.630170e+05	0.012277	0.933326	17.612287	11.600508	4.023253	1.664618	6.927403	21.051146	22.337731	13.000339
83	151713.557862	21772.574976	2.836770e+04	5.79588e+05	23522.564403	47064.003609	139710.950613	5.93030e+05	1.946206e+06	5.610506e+05	3.527399	0.106209	0.602736	13.474087	5.429428	1.093778	3.240201	13.060706	45.200434	15.044051

Ces résultats sont assez logiques ; les départements ruraux ont une part plus importante de leur émission issue de l'agriculture, les départements urbains ont plus d'émissions issues du secteur tertiaire, ce qui est lié à la densité plus importante de ces espaces.

Grâce à ces statistiques on progresse dans la connaissance de notre jeu de données et donc de la nature des émissions de C02 en France. Les statistiques descriptives par groupe nous permettent de mieux saisir l'hétérogénéité spatiale de notre phénomène.

Cependant, on resterait limité dans notre capacité à interpréter ces statistiques sans recourir à de l'information annexe : un département pourrait sembler peu polluant simplement parce qu'il est peu peuplé. C'est exactement le type de limite que nous avons levée grâce à l'enrichissement de nos données par une jointure avec les données Filosofi (voir notamment le calcul d'une empreinte carbone par habitant dans le chapitre précédent).

4 Restructurer les données

4.1 Principe

Quand on a plusieurs informations pour un même individu ou groupe, on retrouve généralement deux types de structure de données :

- format **wide** : les données comportent des observations répétées, pour un même individu (ou groupe), dans des colonnes différentes
- format **long** : les données comportent des observations répétées, pour un même individu, dans des lignes différentes avec une colonne permettant de distinguer les niveaux d'observations

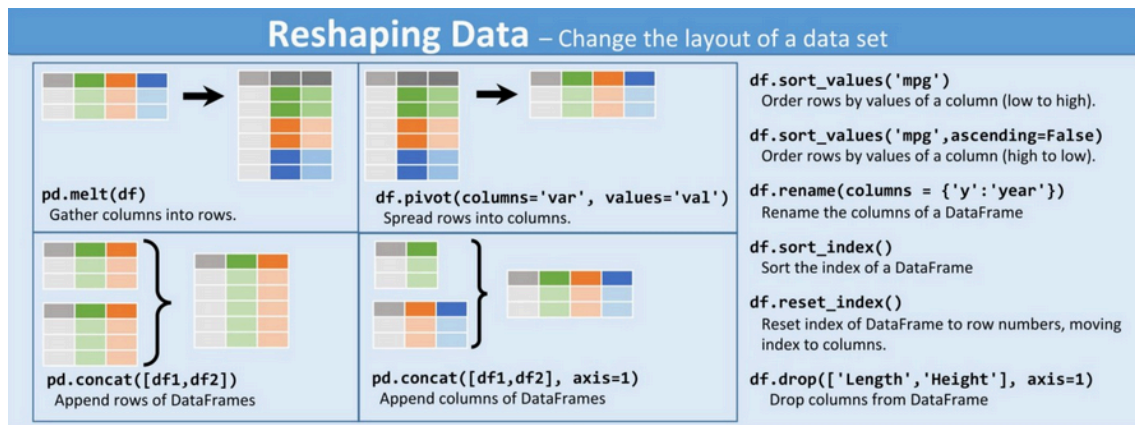
Un exemple de la distinction entre les deux peut être pris à l'ouvrage de référence d'Hadley Wickham, *R for Data Science*:

country	year	cases	country	1999	2000
Afghanistan	1999	745	Afghanistan	745	2666
Afghanistan	2000	2666	Brazil	37737	80488
Brazil	1999	37737	China	212258	213766
Brazil	2000	80488			
China	1999	212258			
China	2000	213766			

table4

Figure 2. – Données *long* et *wide* (Source: *R for Data Science*)

L'aide mémoire suivante aidera à se rappeler les fonctions à appliquer si besoin :



Le fait de passer d'un format *wide* au format *long* (ou vice-versa) peut être extrêmement pratique car certaines fonctions sont plus adéquates sur une forme de données ou sur l'autre.

En règle générale, avec **Python** comme avec **R**, **les formats *long* sont souvent préférables**. Les formats *wide* sont plutôt pensés pour des tableurs comme **Excel** ou on dispose d'un nombre réduit de lignes à partir duquel faire des tableaux croisés dynamiques.

4.2 Exercice d'application

Les données de l'ADEME, et celles de l'Insee également, sont au format *wide*. Le prochain exercice illustre l'intérêt de faire la conversion *long* → *wide* avant de faire un graphique avec la méthode **plot** vue au chapitre d'introduction

💡 Exercice 2: Restructurer les données : wide to long

1. Créer une copie des données de l'ADEME en faisant `df_wide = emissions_wide.copy()`
2. Restructurer les données au format *long* pour avoir des données d'émissions par secteur en gardant comme niveau d'analyse la commune (attention aux autres variables identifiantes).
3. Faire la somme par secteur et représenter graphiquement
4. Garder, pour chaque département, le secteur le plus polluant

5 Formater des tableaux de statistiques descriptives

Un *dataframe* **Pandas** est automatiquement mis en forme lorsqu'il est visualisé depuis un *notebook* sous forme de table HTML à la mise en forme minimaliste. Cette mise en forme est pratique pour voir les données, une tâche indispensable pour les *data scientists* mais ne permet pas d'aller vraiment au-delà.

Dans une phase exploratoire, il peut être pratique d'avoir un tableau un peu plus complet, intégrant notamment des visualisations minimalistes, pour mieux connaître ses données. Dans la phase finale d'un projet, lorsqu'on communique sur un projet, il est avantageux de disposer d'une visualisation attractive. Pour ces deux besoins, les sorties des *notebooks* sont une réponse peu satisfaisante, en plus de nécessiter le *medium* du *notebook* qui peut en rebuter certains.

Heureusement, le tout jeune *package* **great_tables** permet, simplement, de manière programmatique, la création de tableaux qui n'ont rien à envier à des productions manuelles fastidieuses faites dans **Excel** et difficilement répliquables. Ce *package* est un portage en **Python** du *package* **GT**. **great_tables** construit des tableaux *html* ce qui offre une grande richesse dans la mise en forme et permet une excellente intégration avec **Quarto**, l'outil de publication reproductible développé par L'exercice suivant proposera de construire un tableau avec ce *package*, pas à pas. Il nécessite d'installer, au préalable, le *package* **great_tables**:

```
!pip install great_tables --quiet
```

Afin de se concentrer sur la construction du tableau, les préparations de données à faire en amont sont données directement. Nous allons repartir du jeu de données **emissions_merged**, construit au chapitre précédent lors du calcul de l'empreinte carbone par habitant, qui a l'aspect suivant:

Pour être sûr d'être en mesure d'effectuer le prochain exercice, voici le *dataframe* nécessaire pour celui-ci

```
emissions_totales_commune = emissions.sum(axis = 1, numeric_only = True)

emissions_merged = (
    emissions.assign(emissions = emissions_totales_commune)
    .reset_index()
    .merge(filosofi, left_on = "INSEE commune", right_on = "CODGEO")
)
emissions_merged = emissions_merged.loc[emissions_merged['POPULATION'] > 0]
emissions_merged['empreinte'] = emissions_merged['emissions']/emissions_merged['POPULATION']
emissions_merged['empreinte'] = emissions_merged['empreinte'].astype(float)
```

```
emissions_table = (
    emissions_merged
    .rename(columns={"dep_y": "dep", "POPULATION": "population", "NIVVIE_MEDIAN": "revenu"})
    .groupby("dep")
    .agg({"empreinte": "sum", "revenu": "median", "population": "sum"}) #pas vraiment le revenu
    .reset_index()
    .sort_values(by = "empreinte")
)
```

Dans ce tableau nous allons intégrer des barres horizontales, à la manière des exemples présentés [ici](#). Cela se fait en incluant directement le code *html* dans la colonne du *DataFrame*

```
def create_bar(prop_fill: float, max_width: int, height: int, color: str = "green") → str:
    """Create divs to represent prop_fill as a bar."""
    width = round(max_width * prop_fill, 2)
    px_width = f"{width}px"
    return f"""
    <div style="width: {max_width}px; background-color: lightgrey;">\
      <div style="height:{height}px;width:{px_width};background-color:{color};"></div>\
    </div>\
    """

colors = {'empreinte': "green", 'revenu': "red", 'population': "blue"}

for variable in ['empreinte', 'revenu', 'population']:
    emissions_table[f'raw_perc_{variable}'] = emissions_table[variable]/
    emissions_table[variable].max()
    emissions_table[f'bar_{variable}'] = emissions_table[f'raw_perc_{variable}'].map(
        lambda x: create_bar(x, max_width=75, height=20, color = colors[variable])
    )
```

Nous ne gardons que les 5 plus petites empreintes carbone, et les cinq plus importantes.

```
emissions_min = emissions_table.head(5).assign(grp = "5 départements les moins
pollueurs").reset_index(drop=True)
emissions_max = emissions_table.tail(5).assign(grp = "5 départements les plus
pollueurs").reset_index(drop=True)

emissions_table = pd.concat([
    emissions_min,
    emissions_max
])
```

Enfin, pour pouvoir utiliser quelques fonctions pratiques pour sélectionner des colonnes à partir de motifs, nous allons convertir les données au format **Polars**

```
import polars as pl
emissions_table = pl.from_pandas(emissions_table)
```

💡 Exercice 5: Un beau tableau de statistiques descriptives (exercice libre)

En prenant comme base ce tableau

```
GT(emissions_table, groupname_col="grp", rowname_col="dep")
```

construire un tableau dans le style de celui ci-dessous

```
# Start from here
from great_tables import GT
GT(emissions_table, groupname_col="grp", rowname_col="dep")
```

	empreinte	revenu	population	raw_perc_empreinte	bar_empreinte	raw_perc_revenu	bar_revenu	raw_perc_population	bar_population
5 départements les moins pollueurs									
92	142.01712816633915	36310.0	0.06574732880870661455	1.0	0.6326234356093262				
93	147.9911027308174	21820.0	0.1080911936246460305	0.6009363811622143	0.6515878553391433				
94	196.38841932390835	27940.0	0.1426728230470535274	0.7694849903607821	0.5455382727330075				

	empreinte	revenu	population	perc_empreinte	bar_empreinte	bar_revenu	bar_population	bar_population
90	895.5043935192521	29170.0	0104295486563765511		0.8033599559350041		0.05362177826799228	
83	922.1520421556788	26130.0	01050427191994636078		0.7196364637840815		0.4279293555866931	
5 départements les plus pollueurs								
52	13068.456519747897	24610.0	0688453640180125334		0.6777747177086202		0.06435569183009097	
55	13651.074978199267	25340.0	087094207400370646		0.6978793720738089		0.06892781294026117	
51	14401.966686862248	28180.0	063874783963581239		0.7760947397411182		0.21527315546702808	
21	15405.932865409246	27090.0	0484126737931792529		0.7460754613054255		0.2064890552389764	
77	18288.713383716273	28910.0	1468108	1.0	0.7961993941063068		0.5612816772982469	

Le tableau à obtenir:

Empreinte carbone						
Premières statistiques descriptives à affiner						
Empreinte			Revenu médian		Population	
Empreinte carbone	(%)*		Revenu	(%)*	Population	(%)*
5 départements les moins pollueurs						
92	14.20	0.8%	36.3K	100.0%	1.65M	63.3%
93	14.80	0.8%	21.8K	60.1%	1.70M	65.2%
94	19.64	1.1%	27.9K	76.9%	1.43M	54.6%
90	89.55	4.9%	29.2K	80.3%	140.25K	5.4%
83	92.22	5.0%	26.1K	72.0%	1.12M	42.8%
5 départements les plus pollueurs						
52	1,306.85	71.5%	24.6K	67.8%	168.33K	6.4%
55	1,365.11	74.6%	25.3K	69.8%	180.29K	6.9%
51	1,440.20	78.7%	28.2K	77.6%	563.08K	21.5%
21	1,540.59	84.2%	27.1K	74.6%	540.10K	20.6%
77	1,828.87	100.0%	28.9K	79.6%	1.47M	56.1%
<i>*Note: Le revenu médian présenté ici est une approximation du revenu médian du département.</i>						
<i>Lecture: Les colonnes (%) présentées ci-dessus sont rapportées à la valeur maximale de la variable</i>						
<i>Source: Calculs à partir des données de l'Ademe</i>						

Grâce à celui-ci, on peut déjà comprendre que notre définition de l’empreinte carbone est certainement défaillante. Il apparaît peu plausible que les habitants du 77 aient une empreinte 500 fois supérieure à celle de Paris intra-muros. La raison principale ? On n’est pas sur un concept d’émissions à la consommation mais à la production, ce qui pénalise les espaces industriels ou les espaces avec des aéroports...

Pour aller plus loin sur la construction de tableaux avec `great_tables`, vous pouvez répliquer cet [exercice](#) de production de tableaux électoraux que j’ai proposé pour un cours de R avec `gt`, l’équivalent de `great_tables` pour R.

Nous avons désormais fait le tour des principales fonctionnalités de `Pandas` pour manipuler et restructurer des données. Le [dernier chapitre](#) de cette partie propose de prendre un peu de recul sur les limites de la syntaxe `Pandas` et de découvrir les écosystèmes alternatifs qui existent pour aller au-delà.

Informations additionnelles

i Environnement Python

Ce site a été construit automatiquement par le biais d'une action [Github](#) utilisant le logiciel de publication reproductible [Quarto](#) (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'[uv](#). Le fichier [pyproject.toml](#) utilisé pour construire cet environnement est disponible sur le dépôt [linogaliana/python-datascientist](#)

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette",
    "contextily=1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal=3.11.4",
    "graphviz=0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
    "ipykernel≥6.29.5",
    "jupyter≥1.1.1",
    "jupyter-cache≥1.0.0",
    "kaleido≥0.2.1",
    "langchain-community≥0.3.27",
    "loguru=0.7.3",
    "markdown≥3.8",
    "nbclient≥0.10.0",
    "nbformat≥5.10.4",
    "nltk≥3.9.1",
    "pandas≥3.0",
    "pip≥25.1.1",
    "plotly≥6.1.2",
    "plotnine≥0.15",
    "polars≥1.8.2",
    "pyarrow≥17.0.0",
    "pynsee≥0.1.8",
    "python-dotenv≥1.0.1",
    "python-frontmatter≥1.1.0",
    "pywaffle≥1.1.1",
    "requests≥2.32.3",
    "scikit-image≥0.24.0",
    "scikit-learn≥1.8.0",
    "scipy≥1.13.0",
    "seaborn≥0.13.2",
    "selenium<4.39.0",
    "spacy≥3.8.4",
    "webdriver-manager≥4.0.2",
    "wordcloud=1.9.3",
]
```

```
[tool.uv.sources]
cartiflette = { git = "https://github.com/inseeFrLab/cartiflette" }
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'", },
  { index = "geospatial-wheels", marker = "sys_platform == 'win32'", },
]

[[tool.uv.index]]
name = "geospatial-wheels"
url = "https://nathanjmcDougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
  "nb-clean ≥ 4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et `packages`), se reporter à la documentation d'`uv`.

Bibliographie